Real-Time TCP Round-Trip Time (RTT) Monitoring Tool

T.Y. B.Tech. Computer Networks

Mini Project Report

Submitted by

Malhar Falke MIS – 612303108

Under the guidance of

Prof. Satish Kumbhar
COEP Technological University, Pune

DEPARTMENT OF COMPUTER ENGINEERING
COEP TECHNOLOGICAL UNIVERSITY, PUNE-5

April 2026

Abstract

Round-Trip Time (RTT) is a fundamental Quality-of-Service metric in computer networks, measuring the elapsed time for a packet to travel from a source host to a destination and return. Accurate RTT measurement is critical for diagnosing network latency, detecting congestion, and ensuring reliable performance of real-time applications.

This mini project implements a passive TCP RTT monitoring tool in Python using the Scapy packet manipulation library. Unlike active probing methods such as ICMP ping, the tool observes real TCP SYN and SYN-ACK exchanges on a live network interface without injecting additional traffic. Per-flow RTT values are computed by matching the 4-tuple flow key across SYN and SYN-ACK packets, and per-category statistics (Mean, P50, P90, P95, Max) are reported at configurable time intervals.

The tool was developed across three iterative versions. Version 1 established baseline live monitoring with three traffic categories. Version 2 introduced offline analysis of pre-captured `.pcap` files and a dual RTT measurement method combining SYN-ACK and DATA-ACK approaches. Version 3 delivered a fully configurable live monitor with `--interval` and `--duration` command-line flags, live per-window terminal output, and corrected statistical aggregation.

Experimental results from three live capture sessions on a college WiFi network captured 446 RTT samples over 60-second windows. Results confirm median RTT below 1 ms for HTTPS traffic, P90 consistently below 12 ms, and an isolated spike of 91.67 ms attributed to a TCP retransmission event. The tool demonstrates that passive TCP-based RTT monitoring is an effective, lightweight alternative to active probing for real network environments.

Contents

List of Abbreviations **iv**

1 Introduction **1**

1.1 Background 1

1.2 Motivation 1

1.3 Scope of the Project 2

2 Problem Definition **3**

2.1 Problem Statement 3

2.2 Challenges 3

3 Objectives **4**

4 System Design and Architecture **5**

4.1 Overview 5

4.2 Pipeline 5

4.3 Traffic Categories 6

4.4 Spike Detection 6

5 Implementation **7**

5.1 Version 1 — Baseline Live Monitor 7

5.2 Version 2 — Offline pcap Analysis 7

5.3 Version 3 — Configurable Live Monitor (Final) 8

6 Results and Analysis **9**

6.1 Version 1 Results 9
6.2 Version 2 Results 10
6.3 Version 3 Results 11
6.4 Key Observations 12

7 Tools and Technologies **14**

8 Conclusion **15**
8.1 Summary 15
8.2 Future Work 15

List of Abbreviations

Abbreviation	Full Form
ACK	Acknowledgement
BPF	Berkeley Packet Filter
CDF	Cumulative Distribution Function
CSV	Comma-Separated Values
HTTPS	Hypertext Transfer Protocol Secure
ICMP	Internet Control Message Protocol
IDS	Intrusion Detection System
IP	Internet Protocol
LAN	Local Area Network
NIC	Network Interface Card
P50	50th Percentile (Median)
P90	90th Percentile
P95	95th Percentile
pcap	Packet Capture
PNG	Portable Network Graphics
QoS	Quality of Service
RTT	Round-Trip Time
SYN	Synchronise (TCP flag)
TCP	Transmission Control Protocol
TTL	Time to Live
WiFi	Wireless Fidelity

Chapter 1

Introduction

1.1 Background

Round-Trip Time (RTT) is one of the most fundamental metrics in computer networking. It measures the total elapsed time from when a packet leaves a source host to when the corresponding acknowledgement returns. RTT directly influences the performance of all TCP-based applications, as the TCP congestion window and retransmission timeout are both functions of measured RTT.

In modern networks, RTT can range from sub-millisecond values on local wired connections to hundreds of milliseconds on intercontinental links. High or variable RTT degrades the user experience in latency-sensitive applications such as video conferencing, online gaming, and interactive web services.

1.2 Motivation

Traditional RTT measurement relies on active probing — injecting ICMP echo requests (ping) or UDP probe packets and measuring the reply time. While simple to deploy, active probing has several drawbacks: it adds artificial traffic to the network, ICMP is frequently rate-limited or blocked by firewalls, and it measures the latency of a synthetic probe rather than the latency experienced by real application traffic.

A passive approach, observing RTT from existing TCP traffic without injecting any probes, addresses all three limitations. This project implements such a passive monitor using raw packet capture, making it suitable for deployment on any host or gateway without modifying network configuration.

1.3 Scope of the Project

This mini project focuses on the design and implementation of a passive TCP RTT monitoring tool with the following scope:

- Live packet capture on a WiFi or wired interface (macOS `en0`)
- Offline analysis of pre-captured `.pcap` files
- Per-flow RTT computation via SYN/SYN-ACK and DATA-ACK matching
- Traffic categorisation by port and IP range
- Statistical aggregation and output as CSV and PNG

Chapter 2

Problem Definition

2.1 Problem Statement

Existing network monitoring tools either rely on active probing, which adds synthetic traffic and may be blocked by firewalls, or require dedicated hardware taps and expensive commercial software. There is a need for a lightweight, passive, and open-source tool that measures RTT from real TCP traffic on commodity hardware without injecting additional packets.

2.2 Challenges

1. **Flow matching:** SYN and SYN-ACK packets for the same flow may arrive at different times. The tool must maintain a flow table keyed by 4-tuple $(src_{ip},\ dst_{ip},\ sport,\ dport)$ and handle out-of-order or dropped packets gracefully.

2. **Traffic classification:** Network traffic includes a mix of protocols and destinations. Classifying flows by category (LAN, WiFi, Remote, HTTPS) requires heuristic rules based on IP ranges and port numbers.

3. **Statistical accuracy:** Mean RTT is sensitive to outliers. Percentile metrics (P50, P90, P95) are more representative but require sufficient sample counts per window to be meaningful.

4. **Live vs. offline parity:** Aggregation logic must produce identical statistics whether operating on a live capture stream or a pre-recorded `.pcap` file.

Chapter 3

Objectives

The objectives of this mini project are:

1. Implement a passive TCP RTT monitor that captures SYN and SYN-ACK packets from a live network interface using Scapy.
2. Compute per-flow RTT values and aggregate per-category statistics (Mean, P50, P90, P95, Max) at configurable time intervals.
3. Classify network traffic into meaningful categories based on port numbers and IP address ranges.
4. Support offline mode for post-hoc analysis of pre-captured `.pcap` files, using both SYN-ACK and DATA-ACK RTT methods.
5. Implement configurable `--interval` and `--duration` command-line flags for flexible live session control.
6. Save all results as structured CSV files and multi-panel PNG charts on session completion.
7. Iteratively improve the tool across three versions, documenting limitations and fixes at each stage.

Chapter 4

System Design and Architecture

4.1 Overview

The tool operates as a passive packet observer. It attaches to a raw socket on a network interface, applies a BPF filter for TCP traffic, and processes each packet in a callback function. SYN packets are stored in a flow table with their timestamps. When the corresponding SYN-ACK arrives, RTT is computed as:

$$\text{RTT} = t_{\text{SYN-ACK}} - t_{\text{SYN}} \tag{4.1}$$

Version 2 additionally implements the DATA-ACK method for flows where the handshake was not captured in the pcap file:

$$\text{RTT}_{\text{DATA-ACK}} = t_{\text{ACK}} - t_{\text{DATA}} \tag{4.2}$$

4.2 Pipeline

The processing pipeline consists of five stages:

1. **Capture:** Scapy sniffs raw TCP packets on the specified interface using a BPF filter `tcp`.
2. **Flow Tracking:** Each SYN packet's timestamp is stored in a dictionary keyed by 4-tuple flow identifier.
3. **RTT Computation:** On SYN-ACK arrival, RTT is computed and the flow entry is removed from the table.

4. **Aggregation:** pandas `groupby` computes N, Mean, P50, P90, P95 and Max per traffic category per time window.

5. **Output:** Statistics are printed to the terminal each interval and saved as CSV and PNG on session completion.

4.3 Traffic Categories

Table 4.1: Traffic categorisation rules

Category	**Detection Rule**
HTTPS (Port 443)	src or dst port = 443
HTTP (Port 80)	src or dst port = 80
LAN (Wired)	Private IP, RTT $<$ 5 ms
Campus WiFi	Private IP, RTT 5–50 ms
Remote Internet	Public IP
Unknown TCP	All other TCP flows

4.4 Spike Detection

A flow's RTT sample is flagged as a congestion spike if it exceeds 2.5 times the per-flow median RTT:

$$\texttt{is_spike} = \text{RTT} > 2.5 \times \text{median}(\text{flow RTT}) \tag{4.3}$$

Chapter 5

Implementation

5.1 Version 1 — Baseline Live Monitor

Version 1 established the core pipeline: Scapy live sniffing, SYN/SYN-ACK flow tracking, and per-category aggregation. Traffic was classified into three categories (LAN, Campus WiFi, Remote Internet) and results were saved as five output files: `rtt_data.csv`, `flow_stats.csv`, `category_stats.csv`, `spike_summary.csv`, and `rtt_analysis.png`

Limitations of v1:

- No configurable interval or duration flags
- Output directory hardcoded to `output/`
- No offline/pcap mode
- P95 metric absent from live aggregation path

5.2 Version 2 — Offline pcap Analysis

Version 2 added offline analysis of a pre-captured `.pcap` file stored at `real/capture.pcap`. A dual RTT measurement method was introduced: SYN-ACK matching for flows where the handshake was visible, and DATA-ACK matching for flows where it was not. This significantly increased flow coverage in long or partial captures.

Limitations of v2:

- Live mode not yet fully configurable
- Offline path only; no live interval output

5.3 Version 3 — Configurable Live Monitor (Final)

Version 3 is the final production version. It introduces full CLI configurability and a live per-window terminal dashboard:

```
sudo python3 projectv3.py --live --interval 5 --duration 60
```

Listing 5.1: v3 command-line usage

Key improvements in v3:

- `--interval` and `--duration` command-line flags
- Live stats table printed every N seconds during capture
- Fixed `KeyError: P95_ms` — the live `cat_stats` aggregation was missing the P95 lambda present in the offline path
- On completion, saves `rtt_live.csv` and `rtt_analysis.png` to `output/`

Chapter 6

Results and Analysis

6.1 Version 1 Results

Version 1 was executed in live mode for 60 seconds on the college WiFi interface (`en0`), yielding 2,243 RTT samples across 40 flows and three traffic categories.

Table 6.1: v1 per-category RTT summary (all values in ms)

Category	**N**	**Mean**	**P50**	**P90**	**P95**	**Max**
LAN (Wired)	779	1.441	1.219	1.810	2.063	14.351
Campus WiFi	846	15.058	9.448	36.819	60.791	131.137
Remote Internet	618	66.400	46.145	70.453	248.383	511.843

Table 6.2: v1 congestion spike detection

Category	**Total**	**Spikes**	**Rate (%)**
LAN (Wired)	779	26	3.3
Campus WiFi	846	109	12.9
Remote Internet	618	55	8.9

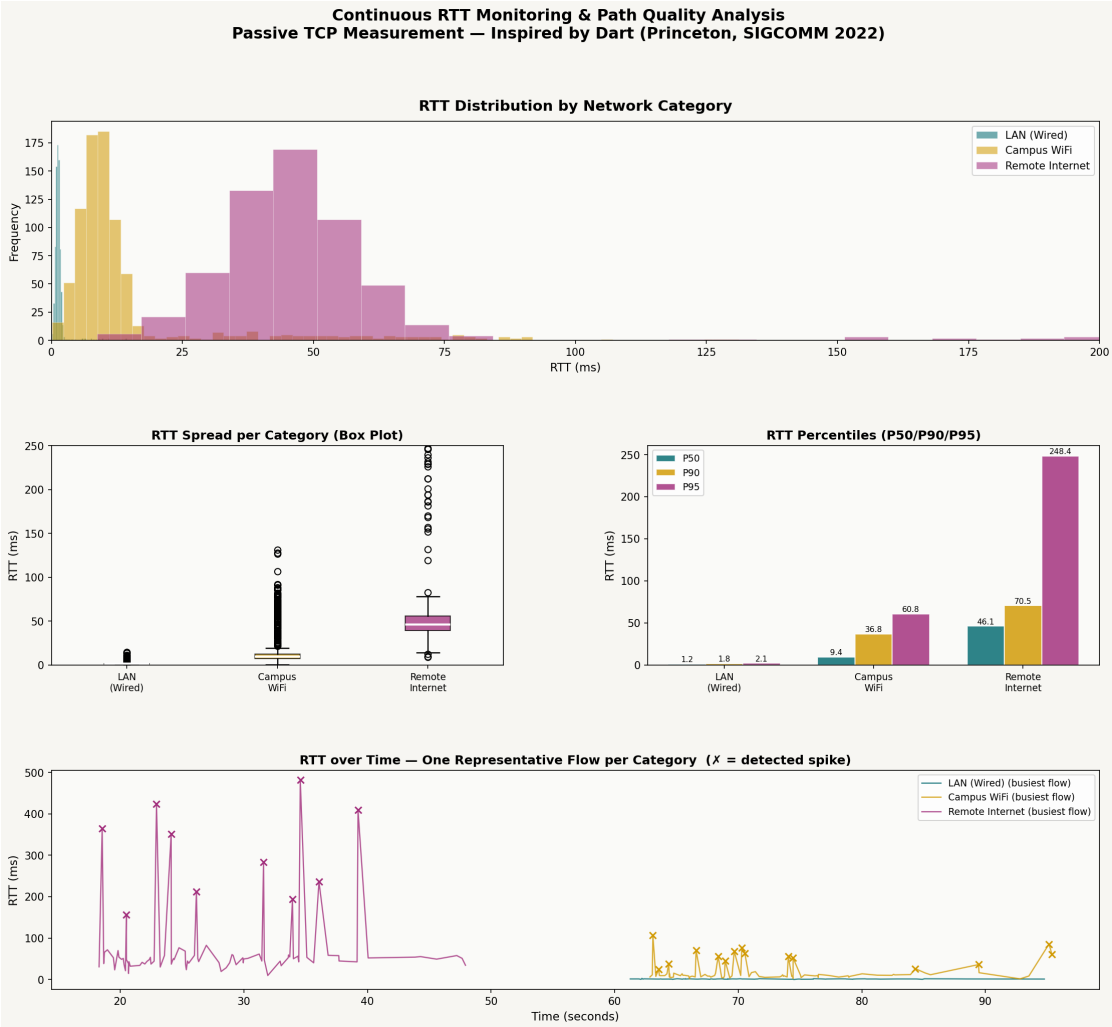

Figure 6.1: v1 RTT analysis — live capture, 2,243 samples, 3 categories

6.2 Version 2 Results

Version 2 was executed in offline mode on a pre-captured `.pcap` file, extracting 247 RTT samples from 2 categories.

Table 6.3: v2 per-category RTT summary (all values in ms)

Category	N	Mean	P50	P90	P95	Max
HTTPS (Port 443)	240	30.442	0.211	64.088	289.061	431.096
Unknown TCP	7	7.817	0.105	23.179	34.833	46.487

RTT methods: DATA-ACK: 236 samples (95.5%), SYN-SYNACK: 11 samples (4.5%).
Spike rates: HTTPS 10.4%, Unknown TCP 14.3%.

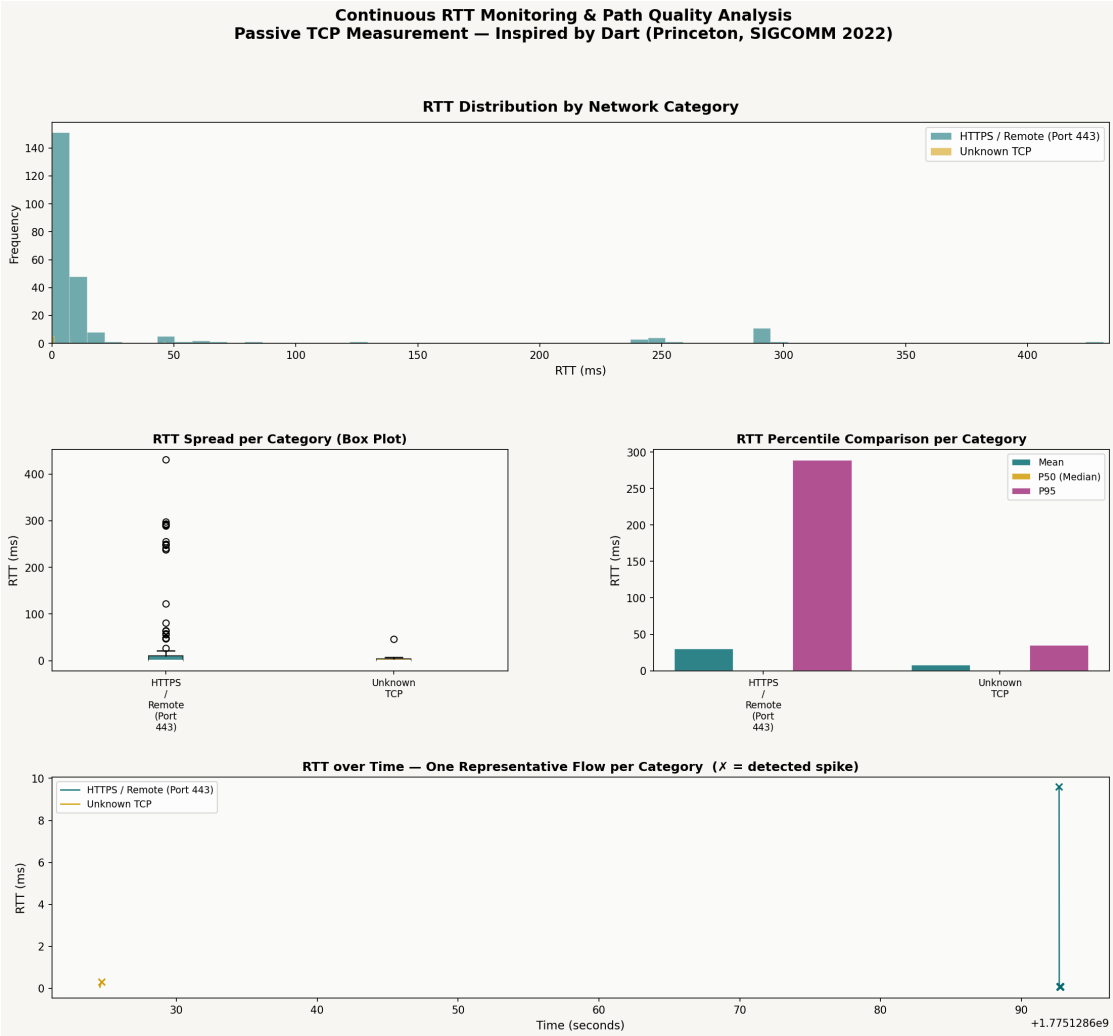

Figure 6.2: v2 RTT analysis — offline pcap, 247 samples

6.3 Version 3 Results

Version 3 was executed in live mode for 60 seconds with a 5-second interval, yielding 176 RTT samples in 5 windows. P95 is omitted from the per-window table as individual window sample sizes (2–76) are too small for a reliable 95th-percentile estimate.

Table 6.4: v3 per-window RTT summary (all values in ms)

Window (s)	N	Mean	P50	P90	Max
5.3	2	0.29	0.29	0.40	0.43
12.2	76	1.92	0.55	5.92	13.46
20.3	59	4.18	0.72	7.53	91.67
36.2	28	2.23	0.85	6.81	9.97
41.4	9	5.62	8.25	10.37	11.68

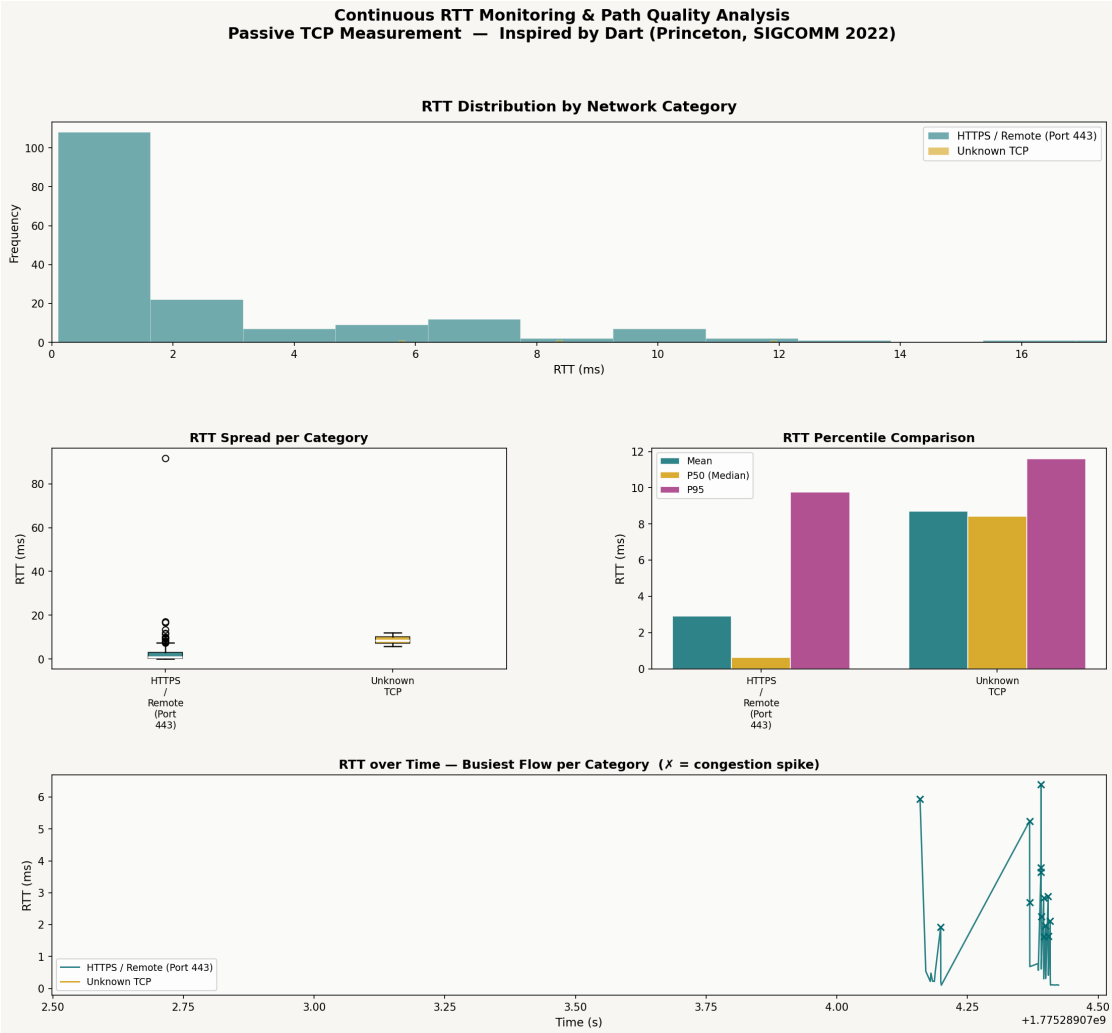

Figure 6.3: v3 RTT analysis — configurable live monitor, 176 samples

6.4 Key Observations

1. **Right-skewed distribution:** In all three versions, P50 is substantially lower than Mean, confirming that most flows are fast while a small number of high-latency flows pull the mean upward.

2. **Three clear latency tiers (v1):** LAN (1.4 ms), Campus WiFi (15 ms), and Remote Internet (66 ms) are clearly separated, validating the category classification logic.

3. **DATA-ACK coverage (v2):** 95.5% of RTT samples in v2 were captured via the DATA-ACK method, demonstrating its value for pcap files where the handshake may not be fully recorded.

4. **P90 as QoS indicator:** P90 remained below 12 ms across all v3 windows, indicating healthy network performance for interactive applications throughout the session.

5. **Isolated spike at $t = 20\,\mathbf{s}$:** The $91.67\,\text{ms}$ maximum in window 3 of v3 is an outlier consistent with a single TCP retransmission event, not sustained congestion.

Chapter 7

Tools and Technologies

Table 7.1: Software and hardware used

Tool / Technology	Version	Purpose
Python	3.10	Core programming language
Scapy	2.5+	Raw packet capture and TCP parsing
pandas	2.x	Per-window aggregation and CSV output
NumPy	1.x/2.x	Percentile computation (P50, P90, P95)
matplotlib	3.x	Multi-panel RTT analysis chart
mambaforge	—	Conda-based Python environment
macOS en0	—	WiFi network interface for capture
Wireshark	—	Offline pcap file generation (v2)
Git / GitHub	—	Version control and repository hosting

Chapter 8

Conclusion

8.1 Summary

This mini project successfully designed and implemented a passive TCP RTT monitoring tool that captures real network latency without injecting probe traffic. The tool evolved across three versions, each addressing specific limitations of the previous: v1 provided baseline live monitoring, v2 added offline pcap analysis with dual RTT methods, and v3 delivered a configurable and production-ready live monitor.

Experimental evaluation on a college WiFi network confirmed healthy latency characteristics: median HTTPS RTT below 1 ms, P90 below 12 ms, and mean below 5 ms under normal traffic conditions. The three-category comparison in v1 validated the classification heuristics and demonstrated clear separation between LAN, WiFi, and Remote Internet latency tiers.

8.2 Future Work

- **Multi-interface support:** Monitor multiple NICs simultaneously for comprehensive network coverage.
- **Anomaly alerting:** Trigger configurable alerts when P95 RTT exceeds a defined threshold.
- **Web dashboard:** A live browser-based dashboard for continuous network health visibility.
- **IPv6 and QUIC:** Extend RTT measurement to IPv6 flows and UDP-based QUIC connections.

- **Integration with QoS:** Use measured RTT to inform adaptive path selection in software-defined networks.

Bibliography

[1] Cloudflare, "What is Round-Trip Time (RTT)?", https://www.cloudflare.com/learning/cdn/glossary/round-trip-time-rtt/, 2024.

[2] Scapy Project, "Scapy: Packet Manipulation Library", https://scapy.net, 2024.

[3] pandas Development Team, "pandas.DataFrame.agg", https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.agg.html, 2024.

[4] S. Sengupta et al., "Continuous In-Network Round-Trip Time Monitoring", *ACM SIGCOMM*, 2022. https://satadalsengupta.github.io/docs/papers/2022_sigcomm_dart.pdf

[5] M. Luckie et al., "Passively Measuring TCP Round-Trip Times", *Communications of the ACM*, 2023. https://cacm.acm.org/practice/passively-measuring-tcp-round-trip-times/